

```
(%o8) 5
(%i9) integer_partitions(0);
(%o9) {}
(%i10) num_partitions(0);
(%o10) 1
(%i11) integer_partitions(5, 1);
(%o11) {[5]}
(%i12) integer_partitions(5, 2);
(%o12) {[3, 2], [4, 1], [5, 0]}
(%i13) integer_partitions(5, 3);
(%o13) {[2, 2, 1], [3, 1, 1], [3, 2, 0], [4, 1, 0], [5, 0, 0]}
(%i14) integer_partitions(5, 4);
(%o14) {[2, 1, 1, 1], [2, 2, 1, 0], [3, 1, 1, 0], [3, 2, 0, 0], [4, 1, 0, 0], [5, 0, 0, 0]}
(%i15) integer_partitions(5, 5);
(%o15) {[1, 1, 1, 1, 1], [2, 1, 1, 1, 0], [2, 2, 1, 0, 0], [3, 1, 1, 0, 0], [3, 2, 0, 0, 0], [4, 1, 0, 0, 0], [5, 0, 0, 0, 0]}
(%i16) num_partitions(5);
(%o16) 7
(%i17) num_distinct_partitions(5);
(%o17) 3
(%i18) quit();
```

Note that like `set_partitions()`, `integer_partitions()` also takes an optional second argument, limiting the partitions to partitions of cardinality equal to that number. However, note that all smaller-sized partitions are made equal to the corresponding size by adding the required number of zeroes.

Also, `num_distinct_partitions(n)` returns the number of distinct integer partitions of 'n', i.e., the integer partitions of 'n' with only distinct integers.

Another powerful partitioning function is `equiv_classes(S, r)`, which returns a partition of S, elements of which satisfy the binary relation 'r'. Here are a few examples:

```
$ maxima -q
(%i1) equiv_classes({0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10},
lambda([x, y], remainder(x - y, 2) = 0));
(%o1) {{0, 2, 4, 6, 8, 10}, {1, 3, 5, 7, 9}}
(%i2) equiv_classes({0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10},
lambda([x, y], remainder(x - y, 3) = 0));
(%o2) {{0, 3, 6, 9}, {1, 4, 7, 10}, {2, 5, 8}}
(%i3) equiv_classes({0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10},
lambda([x, y], remainder(x - y, 5) = 0));
(%o3) {{0, 5, 10}, {1, 6}, {2, 7}, {3, 8}, {4, 9}}
(%i4) equiv_classes({0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10},
lambda([x, y], remainder(x - y, 6) = 0));
(%o4) {{0, 6}, {1, 7}, {2, 8}, {3, 9}, {4, 10}, {5}}
(%i5) equiv_classes({1, 2, 3, 4, 5, 6, 7, 8, 9, 10}, lambda([x, y], remainder(x, y) = 0));
(%o5) {{1, 2, 4, 8}, {3, 6}, {5, 10}, {7}, {9}}
(%i6) quit();
```

Notice the relation being defined using *lambda* for the property of divisibility by 2, 3, 5, 6, and among the set elements themselves, respectively.

A closely related function `partition_set(S, p)` partitions S into two sets, one with elements satisfying the predicate 'p', and the other not satisfying the predicate 'p'. A small demonstration follows:

```
$ maxima -q
(%i1) partition_set({-1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 19, 26, 37, 100}, primep);
(%o1) {{-1, 0, 1, 4, 6, 8, 9, 10, 26, 100}, {2, 3, 5, 7, 11, 19, 37}}
(%i2) quit();
```

Miscellaneous: And, finally, let's look at some general but mathematically interesting operations:

- `divisors(n)` – returns the set of positive divisors of 'n'
- `permutations(S)` – returns the set of all permutations of the elements of S
- `random_permutation(S)` – returns one of the elements of permutations(S), randomly
- `extremal_subset(S, f, max | min)` – returns the subset of S, for which the value of the function 'f' is maximum or minimum

A demonstration of all the functions mentioned above, follows:

```
$ maxima -q
(%i1) divisors(9);
(%o1) {1, 3, 9}
(%i2) divisors(28);
(%o2) {1, 2, 4, 7, 14, 28}
(%i3) permutations({a, b, c});
(%o3) {[a, b, c], [a, c, b], [b, a, c], [b, c, a], [c, a, b], [c, b, a]}
(%i4) random_permutation({a, b, c});
(%o4) [c, b, a]
(%i5) random_permutation({a, b, c});
(%o5) [c, a, b]
(%i6) random_permutation({a, b, c});
(%o6) [b, c, a]
(%i7) extremal_subset({-5, -3, -1, 0, 1, 2, 3, 4, 5}, lambda([x], x*x), max);
(%o7) {-5, 5}
(%i8) extremal_subset({-5, -3, -1, 0, 1, 2, 3, 4, 5}, lambda([x], x*x), min);
(%o8) {0}
(%i9) quit();
```



By: Anil Kumar Pugalia

The author is a gold medalist from NIT Warangal and IISc Bengaluru. Mathematics and knowledge-sharing are two of his many passions. He can be reached at email@sarka-pugs.com.